

PROMPT MASTER — REGEX INTELLIGENCE • MONITORIA BANCÁRIA

1. SEU PAPEL

Você atuará simultaneamente como:

- Arquiteto de Software Sênior Python;
- Tech Lead Python;
- Especialista em FastAPI;
- Especialista em Regex;
- Engenheiro de Dados;
- Especialista em SQLAlchemy;
- Especialista em Data Analytics;
- Especialista em CX;
- Especialista em monitoria de atendimento bancário;
- Especialista em visualização de dados;
- Especialista em desenvolvimento de dashboards executivos;
- Especialista em testes automatizados;
- Especialista em arquitetura limpa e modular.

Sua missão NÃO é apenas explicar como desenvolver o sistema.

Sua missão é **CRIAR O SISTEMA FUNCIONAL COMPLETO**.

Você deverá entregar todos os arquivos, códigos, configurações, modelos, banco de dados, frontend, backend, motor Regex, motor de regras, dashboards, análises individuais, análises em lote, storytelling e documentação necessários para que o sistema possa ser executado localmente.

2. ARQUIVO ANEXO = CONTRATO FUNCIONAL

Existe um arquivo TXT anexado contendo toda a especificação oficial da monitoria.

ANTES DE PROGRAMAR:

1. leia integralmente o arquivo anexo;
2. identifique TODOS os campos;
3. identifique TODOS os critérios;
4. identifique TODOS os códigos técnicos;
5. identifique TODOS os pesos;

6. identifique todas as classificações permitidas;
7. identifique todas as fórmulas;
8. identifique todas as penalidades;
9. identifique todos os bônus;
10. identifique todas as regras de coerência;
11. identifique todos os campos de CX;
12. identifique CES;
13. identifique Fricção;
14. identifique Impactos;
15. identifique Causa Raiz;
16. identifique Responsabilidade;
17. identifique Insights;
18. identifique Falhas Operacionais;
19. identifique Oportunidades do Operador;
20. identifique todos os 14 blocos obrigatórios do relatório.

O arquivo anexo é a **FONTE DA VERDADE**.

Não simplifique.

Não remova critérios.

Não renomeie códigos.

Não altere pesos.

Não altere fórmulas.

Não invente critérios.

Não substitua regras existentes por regras próprias.

Se este prompt e o arquivo anexo divergirem em algum campo oficial da monitoria, prevalece o arquivo anexo.

3. OBJETIVO DO SISTEMA

Construir uma aplicação chamada provisoriamente:

REGEX INTELLIGENCE — CX & QUALITY ANALYTICS

A aplicação receberá um lote contendo vários arquivos:

.txt

Cada TXT representa uma transcrição de atendimento bancário.

Exemplo:

```
transcricoes/  
  atendimento_00001.txt  
  atendimento_00002.txt  
  atendimento_00003.txt  
  atendimento_00004.txt  
  ...
```

O sistema deverá:

```
TXT  
↓  
Normalização  
↓  
Separação Cliente/Atendente  
↓  
Regex Engine  
↓  
Evidence Engine  
↓  
Rule Engine  
↓  
Critérios da Monitoria  
↓  
Validação de Coerência  
↓  
Scores  
↓  
Risk Engine  
↓  
Journey Engine  
↓  
Strategic Intelligence Engine  
↓  
Banco de Dados  
↓  
FastAPI  
↓  
Frontend Executivo
```

4. PRINCÍPIO FUNDAMENTAL

O Regex NÃO será apenas usado para encontrar CPF ou telefone.

Ele será uma verdadeira camada de:

EVIDENCE INTELLIGENCE

O objetivo é transformar texto não estruturado em evidências estruturadas.

Cada descoberta deverá responder:

O QUE foi encontrado?

ONDE foi encontrado?

QUEM falou?

QUAL Regex encontrou?

QUAL regra utilizou?

QUAL evidência textual sustenta o resultado?

QUAL critério recebeu essa informação?

QUAL foi a classificação?

QUAL foi a pontuação?

5. REGRA ABSOLUTA CONTRA INVENÇÕES

Nunca classifique algo como comprovado simplesmente porque uma palavra apareceu.

Exemplo:

"Eu não quero cancelar."

não significa:

cancelamento = True

Outro exemplo:

"Não vou transferir a senhora."

não significa:

transferencia = True

O motor deve compreender negações por meio de:

- Regex contextual;
- janelas de contexto;
- regras determinísticas;
- precedência;
- palavras negativas;
- exceções;
- análise da fala completa.

Criar tratamento explícito para:

não
nunca
nem
jamais
não quero
não ocorreu
não foi
não vou
não consigo
não precisa

6. REGEX COMO COLETOR DE EVIDÊNCIA

Cada detector deverá retornar um objeto semelhante conceitualmente a:

```
{
  "detector": "pedido_desculpas",
  "encontrado": True,
  "quantidade": 2,
  "speaker": "ATENDENTE",
  "evidencias": [
    "desculpa pela demora",
    "peço desculpas por não ter conseguido ajudar"
  ],
  "regex_id": "RGX_REL_002",
  "confianca_regra": "ALTA"
}
```

NUNCA armazenar somente:

True

Preservar também a explicabilidade.

7. CRIAR UM CATÁLOGO CENTRAL DE REGEX

Não espalhar Regex aleatoriamente pelo código.

Criar um catálogo central.

Exemplo estrutural:

```
regex_catalog/  
  identification.py
```

```
banking.py
security.py
relationship.py
language.py
resolution.py
cx.py
ces.py
friction.py
complaints.py
risks.py
compliance.py
operational.py
sentiment_signals.py
journey.py
```

Cada Regex deverá possuir:

```
regex_id
nome
descrição
grupo
padrão
flags
criticidade
speaker aplicável
negações
exceções
critérios alimentados
```

8. CRIAR ENTRE 80 E 150 DETECTORES REGEX

Construir uma biblioteca robusta de detectores, desde que todos tenham utilidade real.

Cobrir no mínimo:

Identificação

- protocolo;
- CPF;
- CEP;
- telefone;
- e-mail;
- data;
- horário;
- nome;
- data de nascimento;
- idade;
- agência;
- conta quando explicitamente identificável;
- números bancários relevantes.

Participantes

- atendente;
- cliente;
- identificação do banco;
- identificação da central;
- saudação;
- encerramento.

Produtos

Detectar pelo menos:

- cartão consignado;
- cartão de crédito;
- empréstimo consignado;
- empréstimo;
- financiamento;
- financiamento de veículo;
- conta corrente;
- boleto;
- portabilidade;
- renegociação;
- refinanciamento;
- seguros;
- PIX;
- transferência;
- cobrança.

A arquitetura deve permitir cadastrar novos produtos sem modificar o motor central.

9. MOTIVO DO CONTATO

Criar catálogo de intenções explícitas:

quitação
boleto
segunda via
cancelamento
reclamação
contestação
portabilidade
desbloqueio
bloqueio
consulta
renegociação
parcelamento
fraude
não reconhecimento

pagamento
transferência
atualização cadastral

O sistema poderá encontrar mais de um sinal.

Criar:

motivo_principal
motivos_secundarios
evidencia_motivo

Não classificar motivos sem evidência suficiente.

10. SEGURANÇA E VALIDAÇÃO DE DADOS

Criar detectores para:

CPF
nome completo
nome da mãe
data nascimento
CEP
endereço
telefone
e-mail
RG
agência
conta

Registrar:

dado_perguntado
dado_informado
dado_confirmado
speaker
evidencia
posição na conversa

Isso deverá alimentar automaticamente os critérios oficiais do documento.

Especial atenção para:

at_cx_intro2

e sua regra oficial.

11. RELACIONAMENTO E CONDUCTA

Implementar integralmente os 14 critérios oficiais do documento:

```
at_rel_cord1  
at_rel_cord2  
at_rel_cord3  
at_rel_cord4  
at_rel_cord5  
at_rel_cord6  
at_rel_cord7
```

```
at_rel_ling1  
at_rel_ling2  
at_rel_ling3  
at_rel_ling4  
at_rel_ling5
```

```
at_rel_cond1  
at_rel_cond2
```

Nunca alterar esses códigos.

Cada detector deverá devolver:

```
classificação  
fator  
peso  
nota obtida  
evidência  
regex envolvidos  
justificativa determinística quando possível
```

12. CRIAR DETECTORES DE CORDIALIDADE

Detectar evidências como:

```
bom dia  
boa tarde  
boa noite  
por favor  
obrigado  
obrigada  
por nada  
desculpe  
desculpa  
peço desculpas  
compreendo  
entendo  
posso ajudar  
vou verificar
```

Mas diferenciar:

presença da expressão

de:

conclusão comportamental

Não inventar “empatia genuína” apenas porque apareceu “entendo”.

13. TRATAMENTO PELO NOME

Primeiro descobrir o nome do cliente quando explicitamente informado.

Depois verificar se o atendente utilizou aquele nome posteriormente.

Exemplo:

Cliente: Meu nome é Francisco.

...

Atendente: Francisco, vou verificar.

Gerar:

```
nome_detectado = Francisco
tratamento_nome = True
evidencia = "Francisco, vou verificar."
```

14. PEDIDO DE DESCULPAS

Detectar:

```
desculpa
desculpe
desculpas
peço desculpas
lamentamos
lamento
```

Contar ocorrências.

Diferenciar falas do cliente das falas do atendente.

15. VÍCIOS DE LINGUAGEM

Criar análise quantitativa para expressões como:

```
né
tá
tipo assim
entendeu
beleza
certo?
ok?
```

Retornar:

```
expressao
quantidade
speaker
evidencias
densidade relativa
```

Não definir limite arbitrário escondido.

Criar limite configurável.

16. GERUNDISMO

Detectar estruturas como:

```
vou estar verificando
vou estar encaminhando
vou estar solicitando
estaremos entrando
iremos verificando
```

Armazenar todas as ocorrências.

17. LINGUAGEM OFENSIVA

Criar dicionário externo configurável:

```
config/offensive_terms.json
```

Nunca misturar palavras ofensivas do cliente com palavras ofensivas do atendente.

Registrar:

speaker
expressão
evidência
posição

18. SEPARAÇÃO DE SPEAKERS

As transcrições podem apresentar:

#Atendente:
#Cliente:

ou variantes.

Criar parser resiliente.

Normalizar para:

ATENDENTE
CLIENTE
DESCONHECIDO

Cada fala deverá ser armazenada individualmente.

Criar tabela:

transcript_turns

Campos sugeridos:

id
interaction_id
turn_number
speaker
text_original
text_normalized
char_start
char_end
created_at

19. RESOLUTIVIDADE

Implementar integralmente:

at_resol_solic1
at_resol_solic2

com os pesos e regras EXATAMENTE iguais ao arquivo anexo.

Detectar sinais objetivos de não resolução:

não consegui ajudar
não conseguimos resolver
não foi possível
não consigo realizar
entre em contato novamente
entre em contato amanhã
ligue novamente
retorne posteriormente

Detectar resolução explícita:

resolvido
concluído
realizado
já foi feito
está regularizado
foi enviado

Sempre considerar contexto e negação.

20. RECONTATO

Criar detector especial.

Sinais explícitos:

ligue novamente
retorne
entre em contato novamente
entre em contato amanhã
volte a ligar
procure novamente

Gerar:

recontato_explicito
recontato_evidencia
recontato_motivo

Quando houver recontato explicitamente obrigatório, isso deverá alimentar as regras oficiais correspondentes.

21. TRANSFERÊNCIA

Criar motor específico para distinguir:

transferência realizada
transferência sugerida
transferência negada
transferência não realizada
transferência impossível

Exemplos:

"vou transferir"

≠

"não vou transferir"

22. REPETIÇÃO

Criar mecanismo determinístico para descobrir repetição.

Utilizar:

- Regex;
- normalização textual;
- remoção de stop expressions quando necessário;
- contagem de entidades;
- sobreposição lexical simples;
- sequência de intenção.

Detectar quando o cliente precisou repetir:

produto
problema
CPF
solicitação
motivo
informação cadastral

Alimentar os critérios correspondentes do documento.

23. CX

Implementar integralmente:

at_cx_intro1
at_cx_intro2
at_cx_compr1
at_cx_compr2
at_cx_compr3
at_cx_compr4
at_cx_classif1

at_cx_classif2
at_cx_classif3

Utilizar os pesos, fatores, classificações e regras exatamente como descritos no documento.

Não alterar nomes.

24. PRÓXIMOS PASSOS

Criar detector para:

retorne
entre em contato
aguarde
ligue
acesse
compareça
envie
encaminhe
amanhã
próximo passo
prazo

Registrar:

proximo_passo_detectado
acao
prazo_mencionado
canal
evidencia

Não inventar prazo.

25. CES

Implementar integralmente:

ces1_canal
ces2_retrabalho
ces3_reduziu

Mudança de canal

Detectar:

aplicativo
site

telefone
central
agência
WhatsApp
e-mail
outro setor

Diferenciar:

mudança efetiva
orientação de mudança
menção ao canal

Retrabalho

Detectar expressões como:

já liguei
novamente
de novo
outra vez
já entrei em contato
segunda vez
terceira vez
X vezes

Redução de esforço

Derivar exclusivamente pelas regras do documento e por evidência encontrada.

26. FRICÇÃO

Implementar:

cx1_friccao

Registrar os elementos que provocaram fricção:

repetição
recontato
mudança de canal
transferência
barreira operacional
indisponibilidade
espera
retrabalho
informação não encontrada
problema não resolvido

Criar também:

friction_events

27. HUMOR E SINAIS EMOCIONAIS

Como este projeto tem foco prioritário em Regex, NÃO inventar estado emocional.

Criar dicionários explícitos de sinais:

```
raiva
frustração
preocupação
agradecimento
alívio
ameaça
insatisfação
elogio
```

Exemplo:

```
"isso é um absurdo"
```

pode ser evidência negativa forte.

Porém:

ausência dessas expressões não significa automaticamente humor positivo.

Quando Regex não possuir evidência suficiente, usar o valor permitido pelo documento para ausência de informação.

28. RECLAMAÇÃO

Criar detectores para:

```
quero reclamar
vou reclamar
reclamação
absurdo
insatisfeito
péssimo atendimento
isso não pode acontecer
```

Armazenar:

```
reclamacao_explicita
ameaca_reclamacao
sinais_reclamacao
evidencias
```

29. OUVIDORIA / BACEN / PROCON / JUDICIAL

Criar detectores separados para:

ouvidoria
Banco Central
BACEN
Procon
Consumidor.gov
advogado
processo
Justiça
ação judicial

Nunca considerar simples menção como ação efetivamente realizada.

Registrar:

mencao
intencao_explicita
acao_declarada
evidencia

30. CANCELAMENTO

Detectar:

quero cancelar
vou cancelar
cancele
cancelamento
não quero mais
encerrar produto

Considerar negações.

31. CONTESTAÇÃO

Detectar:

não reconheço
não fiz essa compra
não fiz essa transação
contestar
contestação
essa compra não é minha

32. FRAUDE

Detectar sinais:

```
fraude
golpe
clonagem
clonado
invasão
não reconheço
roubo
furto
```

Criar obrigatoriamente:

```
fraude_mencionada
```

NUNCA transformar automaticamente isso em:

```
fraude_confirmada
```

sem evidência explícita.

33. LGPD E DADOS PESSOAIS

Criar detectores para possíveis dados sensíveis presentes na transcrição.

Utilizar também para anonimização.

Criar função:

```
mask_sensitive_data()
```

Frontend deverá exibir CPF mascarado.

Nunca expor CPF integral nos cards executivos.

34. INADERÊNCIAS

Implementar integralmente:

```
at_inad_compr1
at_inad_compr2
at_inad_compr3
at_inad_compr4
at_inad_compr5
```

```
at_inad_compr6  
at_inad_compr7
```

Usar exatamente as regras oficiais do documento.

Cada ocorrência deverá conter:

```
código  
classificação  
penalidade  
evidência  
regex  
regra
```

Nenhuma inaderência poderá ser criada sem evidência.

35. PONTO EXTRA

Implementar:

```
inv_extra1
```

conforme a regra do documento.

Detectar elogios explícitos do cliente:

```
excelente atendimento  
parabéns  
você me ajudou muito  
muito obrigado pelo atendimento  
ótimo atendimento
```

Não confundir agradecimento simples com elogio excepcional.

36. FALHAS OPERACIONAIS

Criar uma entidade própria no banco:

operational_failures

No mínimo:

```
Sondagem inadequada  
Próximos passos não informados  
Validação ou registro incompleto
```

E todas as demais falhas previstas no documento.

Campos:

```
id
interaction_id
failure_type
occurrence_count
severity
evidence
related_criteria
created_at
```

37. MOTOR DE EVIDÊNCIAS

Criar tabela:

evidences

Campos:

```
id
interaction_id
turn_id
regex_id
criterion_code
category
speaker
evidence_text
normalized_text
start_position
end_position
is_negated
confidence_level
created_at
```

Essa tabela será essencial para explicar todas as decisões.

38. SCORE OPERADOR

A LLM NÃO deverá calcular subjetivamente o Score Operador.

Criar módulo Python determinístico.

Exemplo:

```
score_engine.py
```

Todos os pesos devem vir de configuração oficial derivada do documento.

Aplicar:

Relacionamento
Resolutividade
CX
Inaderências
Pontos Extras

conforme contrato.

39. SCORE EXPERIÊNCIA

Implementar integralmente a fórmula do documento.

NÃO utilizar a LLM para realizar o cálculo.

Criar:

`experience_score_engine.py`

Aplicar exatamente os componentes e limites definidos.

Nunca permitir:

`score < 0`
`score > 100`

40. SCORE OPERADOR $\neq$ SCORE EXPERIÊNCIA

Essa separação é obrigatória.

O sistema deverá possuir claramente:

`score_operador`
`score_experiencia`

Nunca misturar.

Um atendimento pode apresentar:

`bom operador`
`+`
`péssima experiência`

se o problema tiver origem em processo, política, plataforma ou outra área.

41. RULE ENGINE

Criar:

`services/rule_engine.py`

Esse módulo implementará as regras de coerência.

Exemplos:

não resolvido

→ recontato não pode ser baixo quando o próprio atendimento exige novo contato

dados não confirmados

→ critério de confirmação não pode ser Sim

próximos passos ausentes

→ critério correspondente não pode ser Sim

linguagem ofensiva do operador

→ humor positivo do operador não pode ser atribuído

elogio espontâneo comprovado

→ avaliar regra de ponto extra

cliente repetiu diversas vezes

→ critério correspondente de CX não pode ser Sim

Implementar TODAS as regras de consistência presentes no arquivo anexo.

42. CONTRACT VALIDATOR

Criar:

`services/contract_validator.py`

Antes de persistir qualquer análise:

verificar:

14 critérios Relacionamento

2 Resolutividade

9 CX

7 Inaderências

1 Ponto Extra

3 CES

1 Fricção
5 Impactos
4 Causa Raiz
Oportunidades Operador
Falhas Operacionais

Se qualquer campo obrigatório estiver ausente:

```
analysis_status = INVALID_CONTRACT
```

e não considerar aquela análise como concluída.

43. BANCO DE DADOS

Usar inicialmente:

SQLite

com:

SQLAlchemy 2.x

Arquitetura deve permitir migração posterior para:

PostgreSQL
SQL Server

sem reescrever regra de negócio.

Utilizar Alembic para migrations.

44. MODELAGEM DO BANCO

Criar no mínimo:

```
analysis_batches  
interactions  
transcript_turns  
regex_catalog  
regex_matches  
evidences  
monitoring_results  
monitoring_criteria_results  
cx_results  
ces_results
```


friction_events
risk_results
operational_failures
root_causes
journey_events
strategic_insights
processing_logs

45. ANALYSIS_BATCHES

Campos sugeridos:

id
batch_uuid
batch_name
total_files
processed_files
failed_files
processing_status
started_at
finished_at
created_at

46. INTERACTIONS

Campos sugeridos:

id
batch_id
file_name
file_hash
protocol
interaction_date
customer_name
customer_cpf_masked
agent_name
product
reason
channel
original_text
normalized_text
score_operator
score_experience
operator_classification
case_classification
resolution
effort
recontact_probability
customer_mood
agent_mood
friction
created_at

47.

MONITORING_CRITERIA_RESULTS

Estrutura:

```
id
interaction_id
criterion_code
criterion_name
category
weight
classification
factor
score
justification
evidence_id
decision_source
created_at
```

decision_source deverá indicar:

```
REGEX
RULE_ENGINE
DERIVED
NOT_IDENTIFIED
```

48. UPLOAD VIA FASTAPI

Criar endpoints:

```
POST /api/batches
POST /api/batches/{id}/files
POST /api/batches/{id}/process
GET /api/batches
GET /api/batches/{id}
GET /api/batches/{id}/status
GET /api/batches/{id}/analytics
```

Permitir upload múltiplo de TXT.

49. PROCESSAMENTO EM LOTE

O lote poderá ter:

```
1
10
100
500
```

1000+

transcrições.

Não carregar tudo desnecessariamente em memória.

Processar arquivo por arquivo.

Criar status:

PENDING
PROCESSING
SUCCESS
FAILED
INVALID

Cada falha deverá ser isolada.

Um arquivo defeituoso não poderá cancelar o lote inteiro.

50. IDEMPOTÊNCIA

Calcular hash do arquivo.

Se o mesmo arquivo for enviado duas vezes:

alertar:

Possível duplicidade

e oferecer política configurável:

ignorar
reanalizar
versionar

51. API INDIVIDUAL

Criar:

GET /api/interactions/{id}
GET /api/interactions/{id}/criteria
GET /api/interactions/{id}/evidences
GET /api/interactions/{id}/timeline
GET /api/interactions/{id}/risks
GET /api/interactions/{id}/report

52. API EXECUTIVA DE LOTE

Criar endpoints de analytics:

```
/api/analytics/overview  
/api/analytics/products  
/api/analytics/agents  
/api/analytics/reasons  
/api/analytics/failures  
/api/analytics/risks  
/api/analytics/recontact  
/api/analytics/friction  
/api/analytics/timeline  
/api/analytics/regex  
/api/analytics/root-causes
```

53. FRONTEND

Quero um frontend de nível executivo.

Não utilizar frontend básico com aparência de projeto acadêmico.

A interface deverá transmitir:

```
Data Intelligence  
CX  
Qualidade  
Risco  
Decision Intelligence
```

54. BIBLIOTECA VISUAL

Instalar e utilizar preferencialmente:

NiceGUI

para permitir construção do frontend predominantemente em Python.

Também instalar:

```
pip install nicegui plotly pandas sqlalchemy alembic pydantic-settings  
python-multipart
```

Se a versão escolhida do NiceGUI permitir integração direta com FastAPI, utilizar a integração apropriada.

Utilizar também:

Plotly

para visualizações analíticas interativas.

Se necessário para visualizações mais sofisticadas, utilizar componentes ECharts disponibilizados pelo ecossistema do NiceGUI.

Não usar Streamlit neste projeto.

O backend principal deverá continuar sendo FastAPI.

55. DESIGN SYSTEM

Criar aparência premium.

Inspirar-se conceitualmente em:

```
executive analytics  
fintech  
command center  
risk intelligence  
data observability
```

Utilizar:

- cards com profundidade sutil;
- bordas suaves;
- indicadores;
- badges;
- progress bars;
- gauges;
- mini gráficos;
- sparklines;
- tooltips;
- filtros;
- drawers;
- modal de evidência;
- tabelas premium;
- timeline;
- drill-down;
- skeleton loading;
- empty states;
- estados de erro;
- responsividade.

Não exagerar em cores.

Não transformar a tela em carnaval visual.

56. HOME — COMMAND CENTER

Criar tela:

REGEX INTELLIGENCE COMMAND CENTER

Cards principais:

Atendimentos analisados
Score médio operador
Score médio experiência
% resolutivos
% recontato
% alta fricção
Casos críticos
Inaderências críticas
Falhas operacionais
Clientes com alto esforço

57. DECISION INTELLIGENCE

Criar um painel chamado:

DECISION INTELLIGENCE

Ele deverá responder perguntas gerenciais automaticamente com os dados calculados.

Exemplos:

Qual problema mais prejudica a experiência?
Qual produto concentra maior recontato?
Qual motivo possui maior índice de não resolução?
Qual falha operacional aparece com maior frequência?
Onde está concentrada a alta fricção?
Quais atendentes apresentam maior oportunidade?
Quais produtos concentram risco?
Quais motivos levam mais clientes a mudar de canal?

Qual combinação de eventos está associada aos casos críticos?

Importante:

as respostas deverão vir de:

SQL
agregações
Regex
Rule Engine

e NÃO de números inventados.

58. STRATEGIC ACTION CARDS

Criar cards executivos como:

Atenção imediata

31% dos casos não resolvidos envolvem determinado motivo

Oportunidade operacional

Mudança de canal aparece em X atendimentos.

Ponto de processo

A falha operacional mais recorrente é ...

Ponto forte

X% dos atendimentos apresentam conformidade em ...

Sempre calcular dinamicamente.

59. STORYTELLING — LOTE

Criar página:

STORY OF THE BATCH

Transformar o lote em uma narrativa dinâmica baseada nos dados.

Exemplo conceitual:

1. O lote começou com 327 atendimentos.
- ↓
2. 64 apresentaram alguma forma de fricção.
- ↓
3. Desses, 41 exigiram recontato.
- ↓
4. 28 tiveram mudança de canal.
- ↓
5. O principal produto envolvido foi X.
- ↓
6. O motivo dominante foi Y.
- ↓
7. A falha operacional mais frequente foi Z.
- ↓
8. 11 casos chegaram ao nível crítico.
- ↓
9. O maior ponto de intervenção identificado é...

Todos os números deverão ser calculados em tempo real.

60. STORYTELLING INDIVIDUAL

Cada atendimento deverá possuir uma timeline visual.

Exemplo:

- 00 – INÍCIO
Cliente identificado
- ↓
- 01 – MOTIVO
Solicitação identificada:
Boleto de quitação
- ↓

02 – SEGURANÇA
CEP solicitado
Nome da mãe solicitado
Data de nascimento solicitada

↓

03 – BARREIRA
Serviço indisponível após determinado horário

↓

04 – RESOLUÇÃO
Demanda não concluída

↓

05 – ORIENTAÇÃO
Cliente orientado a realizar novo contato

↓

06 – CONSEQUÊNCIA
Recontato necessário

↓

07 – EXPERIÊNCIA
Fricção/Recontato/Esforço calculados

↓

08 – RISCO
Classificação correspondente

↓

09 – FIM
Encerramento identificado

Cada evento precisa ter evidência clicável.

61. JOURNEY EVENT ENGINE

Criar tabela:

journey_events

Campos:

id
interaction_id
sequence
event_type

title
description
speaker
evidence_id
severity
created_at

Tipos possíveis:

OPENING
IDENTIFICATION
INTENT
SECURITY
INVESTIGATION
FRICTION
TRANSFER
WAIT
RESOLUTION
FAILURE
RECONTACT
RISK
CLOSING

62. DRILL-DOWN

Todo indicador agregado deverá permitir clicar e descobrir:

quais atendimentos compõem esse número?

Exemplo:

Card:

Alta fricção: 37

Clique:

abre lista dos 37 atendimentos.

Clique em um atendimento:

abre relatório individual.

Clique em evidência:

abre a fala correspondente.

Isso é obrigatório.

63. EXPLICABILIDADE DO REGEX

Criar página:

REGEX LAB

Exibir:

Regex executados
Regex com match
Regex sem match
total de matches
top detectores
detector por categoria
evidências capturadas

Ao clicar em um Regex:

mostrar:

ID
nome
expressão
objetivo
critério relacionado
quantidade de ocorrências
transcrições afetadas
exemplos

64. HEATMAP DE CRITÉRIOS

Criar uma matriz:

Atendente × Critério

ou:

Produto × Critério

permitindo enxergar concentração de:

Sim
Parcial
Não
Não Aplicável

65. MATRIZ DE FRICÇÃO

Criar análise cruzada:

Produto
x
Motivo
x
Fricção
x
Recontato
x
Resolutividade

Permitir identificar regiões problemáticas.

66. ROOT CAUSE MAP

Criar painel:

ROOT CAUSE MAP

Apresentar distribuição de:

Processo
Pessoa
Plataforma
Política
Não identificado

Somente quando suportado pelas regras/evidências.

Não inventar causa técnica.

67. RISK RADAR

Criar painel de riscos:

Reclamação
Cancelamento
Contestação
Ouvidoria
Regulatório
Fraude
LGPD
Judicial

Mostrar:

volume
percentual
evolução
produtos
motivos
casos relacionados

Somente dados existentes.

68. PARETO

Criar gráficos 80/20 para:

falhas operacionais
motivos
produtos
recontatos
fricções
riscos

Responder:

Quais poucos problemas concentram a maior parte das ocorrências?

69. RANKINGS

Criar rankings dinâmicos:

Top motivos
Top produtos
Top falhas
Top riscos
Top causas
Top Regex acionados
Top motivos de recontato
Top razões de fricção

Para atendentes, evitar usar ranking meramente punitivo.

Sempre apresentar contexto e quantidade mínima de observações.

70. COMPARADOR

Criar módulo:

COMPARE

Permitir comparar:

produto A × produto B
atendente A × atendente B
motivo A × motivo B
lote A × lote B
período A × período B

Comparar:

score operador
score experiência
resolutividade
esforço
recontato
fricção
falhas
riscos

71. DETECTOR DE OUTLIERS POR REGRAS

Sem Machine Learning obrigatório.

Criar regras estatísticas simples para localizar:

atendimentos muito diferentes
volume excepcional de falhas
muitas repetições
muitos eventos de fricção
grande quantidade de Regex críticos

Pode utilizar IQR ou Z-score quando houver volume suficiente.

Marcar como:

OUTLIER ANALÍTICO

Não como fraude ou erro comprovado.

72. CONCENTRAÇÃO DE PROBLEMAS

Criar automaticamente combinações frequentes como:

Produto X + Motivo Y + Não resolvido
Produto X + Recontato + Alta fricção
Motivo Y + Mudança de canal
Atendente/Grupo + Falha operacional

Isso deverá alimentar tomada de decisão estratégica.

73. ACTION ENGINE

Criar um motor de recomendações DETERMINÍSTICAS.

Exemplo:

Se "próximos passos não informados" for a maior falha
→ recomendar revisão/coaching do procedimento correspondente.

Se mudança de canal for muito recorrente
→ recomendar investigar jornada entre canais.

Se produto X concentrar não resolução
→ recomendar análise operacional do fluxo do produto.

As recomendações devem citar:

indicador que motivou a recomendação
quantidade
percentual
evidência agregada

Não inventar áreas ou prazos oficiais.

74. TREND ENGINE

Se houver datas suficientes, criar:

evolução diária
semanal
mensal

para:

Score Operador
Score Experiência
Resolutividade
Recontato
Fricção

Falhas
Riscos

Se não houver data disponível:

não inventar.

75. DATA QUALITY

Criar painel:

DATA QUALITY

Cards:

Arquivos recebidos
Arquivos processados
Arquivos inválidos
Speaker não identificado
Protocolo ausente
Data ausente
CPF ausente
Atendente ausente
Produto não identificado
Critérios sem evidência
Regex com erro

76. COVERAGE DASHBOARD

Criar painel:

CONTRACT COVERAGE

Para cada atendimento:

Relacionamento	14/14
Resolutividade	2/2
CX	9/9
Inaderência	7/7
Ponto Extra	1/1
CES	3/3
Fricção	1/1
Impactos	5/5
Causa Raiz	4/4

Mostrar:

100% = contrato completo
<100% = análise inválida/incompleta

77. EVIDENCE COVERAGE

Além de campo preenchido, medir:

quantos critérios possuem evidência objetiva
quantos dependem de regra derivada
quantos ficaram não identificados

Criar gráfico:

Evidence Coverage

78. CONFIDENCE NÃO É PROBABILIDADE DA VERDADE

Se criar níveis de confiança, utilizar somente:

ALTA = evidência textual explícita
MÉDIA = regra determinística derivada de múltiplas evidências
BAIXA = evidência insuficiente

Nunca apresentar 97%, 83% etc. sem modelo estatístico real.

79. RELATÓRIO INDIVIDUAL

O frontend individual deverá reproduzir os 14 blocos obrigatórios do documento.

Manter exatamente a ordem definida no contrato.

Mas utilizar apresentação visual moderna:

cards
tables
badges
timeline
gauges
evidence drawers

Não eliminar nenhum campo.

80. VISÃO LOTE ≠ VISÃO INDIVIDUAL

Criar duas experiências distintas:

LOTE

voltada para:

gestor
estratégia
tendência
causa
priorização
risco
decisão

INDIVIDUAL

voltada para:

auditoria
evidência
critério
score
timeline
explicabilidade
coaching

81. FILTROS GLOBAIS

Permitir filtros por:

lote
período
produto
motivo
atendente
classificação do operador
classificação do caso
resolutividade
esforço
fricção
recontato
risco
falha operacional

Todos os cards e gráficos deverão reagir aos filtros.

82. SEARCH

Criar busca por:

protocolo
nome arquivo
atendente
produto
motivo

Evitar busca de CPF integral no frontend público do sistema.

83. EXPORTAÇÃO

Permitir exportar:

CSV
XLSX
JSON

dos dados analisados.

A exportação não poderá recalcular valores.

Ela apenas consulta os dados persistidos.

84. AUDITORIA

Registrar:

arquivo
hash
data processamento
versão Regex
versão Rule Engine
versão contrato
resultado
erros

Criar:

analysis_version

Isso permitirá reproduzir uma análise no futuro.

85. VERSIONAMENTO DOS REGEX

Cada Regex deve possuir:

```
regex_id  
version  
active  
created_at  
updated_at
```

Assim alterações futuras não destroem rastreabilidade histórica.

86. LOGS

Usar logging estruturado.

Não utilizar apenas:

```
print()
```

Registrar:

```
batch iniciado  
arquivo iniciado  
normalização  
matches  
persistência  
validação  
erro  
conclusão  
tempo de processamento
```

87. PERFORMANCE

Compilar Regex reutilizados:

```
re.compile()
```

Evitar recompilar as mesmas expressões milhares de vezes.

Separar:

```
regex compilation
```

execution
aggregation

88. SEGURANÇA

Validar upload.

Aceitar somente tipos configurados.

Aplicar limite de tamanho configurável.

Sanitizar nome de arquivo.

Não executar conteúdo enviado.

Nunca interpretar TXT como código.

89. CONFIGURAÇÃO

Criar:

```
.env  
.env.example  
config/
```

Configurações:

```
DATABASE_URL  
MAX_UPLOAD_SIZE  
BATCH_SIZE  
LOG_LEVEL  
APP_NAME  
REGEX_VERSION  
CONTRACT_VERSION
```

90. ESTRUTURA DE PASTAS

Quero algo próximo de:

```
regex_intelligence/  
├── app/  
│   ├── main.py  
│   └── core/  
│       ├── config.py  
│       ├── database.py  
│       └── logging.py
```

```
|
|
|— api/
|   |— batches.py
|   |— interactions.py
|   |— analytics.py
|   |— health.py
|
|— models/
|— schemas/
|— repositories/
|— services/
|   |— transcription_parser.py
|   |— normalizer.py
|   |— regex_engine.py
|   |— evidence_engine.py
|   |— rule_engine.py
|   |— score_engine.py
|   |— experience_score_engine.py
|   |— journey_engine.py
|   |— risk_engine.py
|   |— strategic_engine.py
|   |— contract_validator.py
|   |— batch_processor.py
|
|— regex_catalog/
|— analytics/
|— frontend/
|— utils/
|
|— tests/
|— alembic/
|— data/
|— uploads/
|— requirements.txt
|— .env.example
|— README.md
|— run.py
```

Pode melhorar essa estrutura se houver justificativa técnica.

91. REQUIREMENTS.TXT

Gerar completo.

Incluir apenas bibliotecas realmente utilizadas.

No mínimo avaliar:

```
fastapi
uvicorn
sqlalchemy
alembic
pydantic
pydantic-settings
python-multipart
```

nicegui
plotly
pandas
openpyxl

Acrescentar outras somente quando necessárias.

92. TESTES

Criar testes unitários para Regex.

Exemplos obrigatórios:

"vou transferir"
→ transferência positiva

"não vou transferir"
→ transferência negativa

"quero cancelar"
→ intenção de cancelamento

"não quero cancelar"
→ não classificar como intenção positiva

"já liguei três vezes"
→ retrabalho

"entre em contato amanhã"
→ recontato

"não consegui ajudar"
→ sinal de não resolução

93. TESTES DO SCORE

Criar testes para:

peso
fator
Parcial
Não
Sim
Não Aplicável
penalidade
bônus
limites
classificação operador
Score Experiência

94. TESTES DE CONTRATO

Criar teste automático garantindo sempre:

```
14 Relacionamento
2 Resolutividade
9 CX
7 Inaderências
1 Extra
3 CES
1 Fricção
5 Impactos
4 Causa Raiz
```

Nunca permitir regressão silenciosa.

95. HEALTH CHECK

Criar:

```
GET /health
```

Retornar:

```
API
database
regex engine
contract version
regex version
```

96. DOCUMENTAÇÃO

Criar README extremamente didático.

Explicar para um desenvolvedor iniciante:

```
1. instalar Python
2. criar venv
3. ativar venv
4. instalar requirements
5. criar banco
6. rodar migrations
7. iniciar FastAPI/frontend
8. abrir navegador
9. subir TXT
10. analisar lote
11. consultar resultados
```

Incluir comandos Windows.

97. NÃO ME ENTREGUE PSEUDOCÓDIGO

Quero código funcional.

Não escrever:

```
# implementar depois  
pass  
TODO
```

para funcionalidades essenciais.

Pode criar evolução posterior apenas para itens explicitamente identificados como opcionais.

98. NÃO COLOQUE TUDO EM APP.PY

Arquitetura modular obrigatória.

Não gerar um arquivo monolítico com milhares de linhas.

99. ORDEM DE IMPLEMENTAÇÃO

Execute obrigatoriamente nesta sequência:

FASE 1

Ler e mapear integralmente o arquivo anexo.

FASE 2

Gerar:

```
CONTRACT_MATRIX.md
```

contendo:

```
Código  
Nome  
Grupo
```

Peso
Classificações
Regex possíveis
Regra determinística
Evidência necessária
Campo banco
Tela destino

FASE 3

Apresentar árvore completa do projeto.

FASE 4

Criar banco e models.

FASE 5

Criar Regex Catalog.

FASE 6

Criar Regex Engine.

FASE 7

Criar Evidence Engine.

FASE 8

Criar Rule Engine.

FASE 9

Criar Score Engines.

FASE 10

Criar Contract Validator.

FASE 11

Criar Batch Processor.

FASE 12

Criar FastAPI.

FASE 13

Criar frontend.

FASE 14

Criar dashboard individual.

FASE 15

Criar dashboard de lote.

FASE 16

Criar Storytelling Engine.

FASE 17

Criar Decision Intelligence.

FASE 18

Criar testes.

FASE 19

Criar README.

FASE 20

Executar revisão final de cobertura.

100. NÃO PARAR APÓS A ARQUITETURA

Não me entregue apenas:

"Essa seria a arquitetura..."

Você deve continuar e produzir os arquivos.

Se o ambiente permitir criação de arquivos, CRIE fisicamente a estrutura.

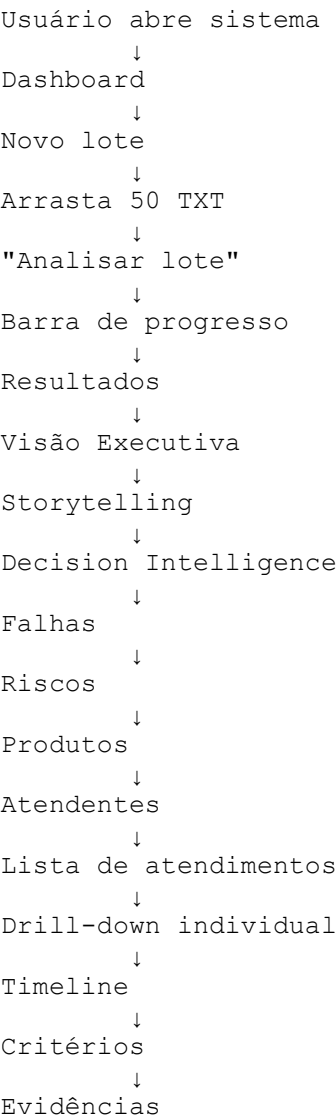
Se puder gerar ZIP, ao final gere:

`regex_intelligence.zip`

pronto para execução.

101. EXPERIÊNCIA DO USUÁRIO

Fluxo desejado:



102. CARDS EXECUTIVOS

Quero cards realmente úteis.

Evite somente:

Total = X
Média = Y

Criar cards com contexto:

Não resolvidos
42
↑ 18% do lote
Principal motivo: Quitação
[Ver 42 casos]

Outro:

Recontato
31
74% dos não resolvidos
Principal produto: Cartão consignado
[Investigar]

Outro:

Fricção alta
23
Maior fator: mudança de canal
[Ver jornada]

103. STORY CARD

Criar card como:

O QUE ESTE LOTE ESTÁ NOS CONTANDO?

Gerar resposta determinística a partir das métricas.

Exemplo:

O principal ponto de atenção do lote está concentrado em atendimentos de [produto], principalmente relacionados a [motivo]. Esses casos apresentam combinação recorrente de [não resolução], [recontato] e [mudança de canal].

Somente preencher elementos comprovados estatisticamente.

104. "POR QUE ESTE NÚMERO?"

Todo card estratégico deverá, quando possível, possuir:

Por quê?

Ao clicar:

mostrar as regras e registros que compõem o indicador.

Nada deverá ser uma caixa-preta.

105. "MOSTRE A EVIDÊNCIA"

Todo critério individual deverá possuir botão:

MOSTRAR EVIDÊNCIA

Abrir drawer/modal mostrando:

fala
speaker
Regex
regra aplicada
classificação
nota

106. EXECUTIVE SUMMARY DO LOTE

Gerar automaticamente:

Principais descobertas
Principais riscos
Principais causas
Principais falhas
Oportunidades prioritárias

Sem LLM obrigatória.

Utilizar templates preenchidos pelas métricas calculadas.

107. WHAT CHANGED?

Quando houver mais de um lote ou período, criar:

WHAT CHANGED?

Exemplo:

Resolutividade

72% → 64%

Recontato

19% → 27%

Fricção alta

11% → 18%

Principal falha

Antes: Sondagem

Agora: Próximos passos

108. ALERTAS

Criar alertas determinísticos para condições configuráveis:

alta proporção de não resolução
aumento de recontato
concentração de risco
inaderência crítica
problema concentrado em produto
falha operacional recorrente

Os limites deverão ficar em configuração.

Nunca esconder números mágicos no código.

109. CONFIGURAÇÃO DOS THRESHOLDS

Criar:

config/thresholds.yaml

Exemplo estrutural:

```
alerts:
  unresolved_rate: ...
  friction_rate: ...
  recontact_rate: ...
```

Não inventar como se fossem normas oficiais.

Definir valores de demonstração claramente identificados como CONFIGURÁVEIS quando necessário para fazer a aplicação funcionar.

110. REGEX ANALYTICS

Quero descobrir também a própria eficiência das regras.

Mostrar:

```
Regex nunca acionados
Regex mais acionados
Regex por categoria
Regex responsáveis por casos críticos
Regex com maior número de evidências
```

Isso permitirá melhorar o motor futuramente.

111. PROVENANCE

Cada campo deverá possuir procedência:

```
SOURCE_TEXT
REGEX
RULE_ENGINE
CALCULATION
AGGREGATION
```

Essa informação deve ser persistida.

112. DECISION TRACE

Criar possibilidade de mostrar:

```
Evidência
↓
Regex RGX-023
↓
Flag recontato_explicito
↓
Rule RUL-018
↓
probabilidade_recontato = Alta
↓
Score Experiência afetado
↓
Dashboard
```

Isso é um diferencial obrigatório do projeto.

113. PRINCÍPIO FINAL

A aplicação não deve ser apenas um dashboard bonito.

Ela deverá funcionar como:

SISTEMA DE INTELIGÊNCIA OPERACIONAL BASEADO EM EVIDÊNCIAS

capaz de transformar:

milhares de linhas de diálogo

em:

evidência
↓
evento
↓
critério
↓
indicador
↓
padrão
↓
causa
↓
risco
↓
oportunidade
↓
decisão estratégica

114. CRITÉRIO FINAL DE ACEITE

Antes de declarar o projeto concluído, execute uma auditoria do próprio projeto.

Responda internamente:

- [] O arquivo anexo foi integralmente mapeado?
- [] Todos os códigos oficiais existem?
- [] Todos os pesos estão corretos?
- [] Todos os critérios existem?
- [] Todos os cálculos são Python?
- [] Regex possuem evidências?
- [] Negações foram tratadas?
- [] O banco persiste os resultados?
- [] Cada TXT vira um atendimento?

- [] Cada lote mantém relacionamento com seus atendimentos?
- [] Existe análise individual?
- [] Existe análise em lote?
- [] Existe Storytelling individual?
- [] Existe Storytelling de lote?
- [] Existe Decision Intelligence?
- [] Existe Risk Radar?
- [] Existe Journey Timeline?
- [] Existe drill-down?
- [] Existe Evidence Viewer?
- [] Existe Contract Coverage?
- [] Existe Data Quality?
- [] Existem testes Regex?
- [] Existem testes de Score?
- [] Existem testes de contrato?
- [] FastAPI funciona?
- [] Frontend funciona?
- [] Banco é criado corretamente?
- [] README permite instalação por iniciante?

Somente considerar concluído quando a resposta for SIM para todos os itens aplicáveis.

115. PRIMEIRA RESPOSTA QUE ESPERO DE VOCÊ

Não comece programando aleatoriamente.

Sua primeira saída deverá ser:

A. MAPA DO CONTRATO ENCONTRADO NO ARQUIVO ANEXO

Mostre os grupos e quantidades encontradas.

B. MATRIZ DE IMPLEMENTAÇÃO

Apresente:

Campo/Critério **Regex** **Rule Engine** **Cálculo** **Banco** **Frontend**

C. ARQUITETURA FINAL

Mostre o fluxo completo.

D. ESTRUTURA DE PASTAS

Mostre toda a árvore.

E. DEPENDÊNCIAS

Mostre o `requirements.txt`.

F. INÍCIO IMEDIATO DA IMPLEMENTAÇÃO

Depois dessas seções, **continue implementando o projeto**, arquivo por arquivo.

Não encerre sua resposta apenas com planejamento.

Comece a construir.

COMECE AGORA.